ENEL300

Electrical and Computer Engineering Professional Skills

Final Design Report

Radio-Controlled Car with Metal Detection and Distance Sensing

April 26, 2026

UNIVERSITY OF CALGARY

Group 23

Dustin Christensen
Nick Mulligan
Brian Oh
Diana Raduta
Andy Xu

Contents

1 Abstract **3**

2 Background **4**

3 Detailed Design **5**

3.1 Electrical Architecture . . . 5
3.2 Metal Detector, Distance Sensing, and Motor Drive Circuits . . . 5
3.3 Power Management . . . 8
3.4 Printed Circuit Board Design . . . 8
3.5 Remote Controller Design . . . 9
3.6 Software Architecture . . . 9
3.7 Software Design . . . 12
3.8 Mechanical Architecture . . . 13
3.9 Mechanical Design . . . 16

4 References **19**

5 Appendices **20**

List of Figures

1 Electrical block diagram for the design . . . 5
2 The metal detection circuit . . . 6
3 The PCB schematic for the metal detection circuit . . . 7
4 Software block diagram for the movement of the design . . . 10
5 Software block diagram for the STM32 (1) . . . 10
6 Software block diagram for the STM32 (2) . . . 11
7 Drawing of the initial design (1) . . . 14
8 Drawing of the initial design (2) . . . 14
9 Drawing of the final design (1) . . . 15
10 Drawing of the final design (2) . . . 15
11 Mechanical block diagram for the initial design . . . 16
12 Mechanical block diagram for the final design . . . 16
13 Implementation of the final design (1) . . . 20
14 Implementation of the final design (2) . . . 21
15 Implementation of the final design (3) . . . 22
16 Schematic for the ESP PCB . . . 23
17 Layout for the ESP PCB . . . 24
18 3D rendering of the ESP PCB . . . 25
19 Schematic for the STM32 PCB . . . 26
20 Layout for the STM32 PCB . . . 27
21 3D rendering of the STM32 PCB . . . 28

List of Tables

1 Bill of materials 20

1 Abstract

The purpose of this project was to create a radio-controlled vehicle for a competition at the end of the semester. The requirements of the vehicle were to be able to move wirelessly, detect copper plates, and measure the distance of nearby objects. The final project used skills integrated from mechanical, software, and electrical engineering concepts.

The vehicle used both an STM32 microcontroller and an ESP32 module to create a reliable and efficient control system. Firstly, the ESP32 was controlled through a smartphone webserver interface developed in Visual Studio Code. The ESP32 was responsible for motor control and headlight operation. Then, the STM32 controlled the ultrasonic distance sensing and the metal detection functions. For steering, the vehicle used a tank steering system, where the left and right motors were controlled independently for improved maneuverability.

A custom 3D-printed chassis was designed in Fusion 360, ensuring the ability to mount all major components. For example, the metal detection coil was placed underneath the vehicle, the PCBs were mounted on the body, and the headlights and distance sensor were positioned at the front of the vehicle. This project included two custom PCBs: one for metal detection connected to the STM32, and one for motor control and power management connected to the ESP32. The system was powered by a 12 V battery with battery protection and reverse-polarity protection circuitry.

Key challenges included improving the metal detection system, configuring the STM32 code for proper sensor operation, and ensuring all components fit correctly within the chassis. The completed vehicle successfully demonstrated wireless control, tank steering movement, distance sensing, headlight control, and metal detection. The final design met the main project objectives while providing practical experience in embedded systems, PCB design, programming, and mechanical integration.

2 Background

This RC car was developed using many different ideas and inspirations, but it was mainly influenced by previous projects and common designs used in robotic platforms.

Firstly, the mechanical design was inspired by a previous project completed in ENGG 200. A tank steering system was selected because it is easier to implement than many other steering methods. This design made both the chassis development and the motor control logic simpler. The chassis was created using ideas from several previous designs so that it could efficiently support all required components, including the battery, motors, sensors, and PCB boards.

Furthermore, another major inspiration for the project came from the Garden Project completed in ENGG 200. That group used a webserver to monitor soil moisture levels, which inspired our team to use a webserver to control the RC car instead of using a traditional handheld controller. Rather than using the STM32 to control the motors, an ESP32 was selected for this task. The specific ESP32 module was chosen because it included built-in Wi-Fi, Bluetooth capability, and an onboard antenna, which simplified wireless communication and made implementation easier.

Other inspirations for the car came from the project requirements themselves. Since the ultrasonic sensor was provided and had been used in previous classes, it was the most practical option for implementing the distance sensing system. In addition, the STM32 was also provided, and since workshops were conducted to help students learn how to use it, it was the most effective option for implementing the distance sensing and metal detection circuits without interfering with the motor control code. Lastly, the metal detection circuit was inspired by online research followed by trial and error during development. Many circuit variations were tested before selecting the final design that best suited the vehicle.

3 Detailed Design

3.1 Electrical Architecture

The electrical block diagram is shown in Figure 1.

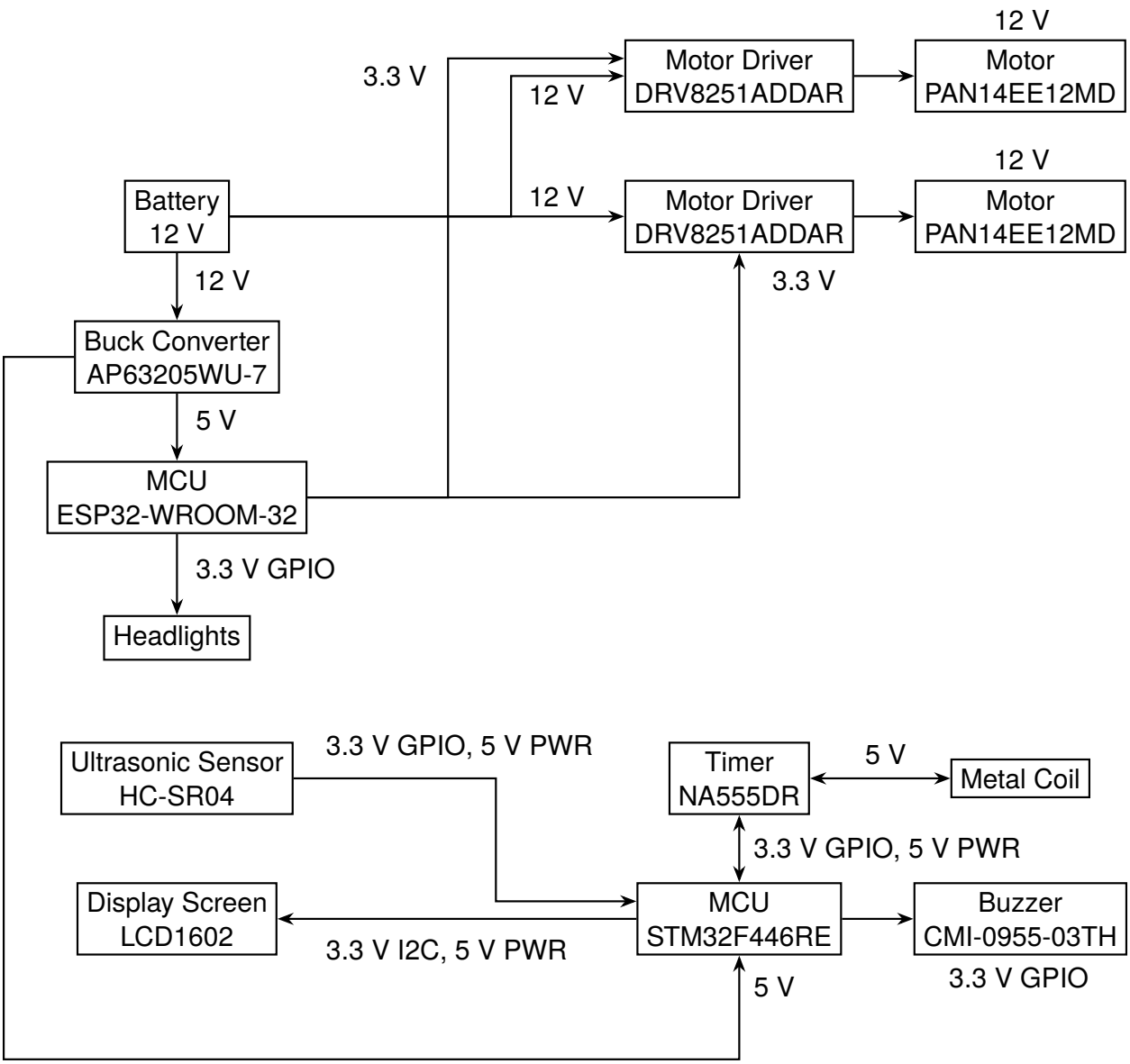

Figure 1: Electrical block diagram for the design

3.2 Metal Detector, Distance Sensing, and Motor Drive Circuits

For the metal detection circuit, many designs were researched before finding the one that we decided to use. The main differences between them were the type of coil used to detect metal and the circuitry used to generate the signal. The different types of coils included transmit/receive coils, coils wrapped around a ferrite core and single coils. As for the generation, the options were between a 555 timer, a transistor setup, and a CD4011 NAND gate. For our first iteration of the metal

detector, we went with a 555 timer and a single coil design because all of the components were readily available in the school, and the design for the circuit was the simplest of all the options. The metal detection circuit obtained from [2] is shown in Figure 2.

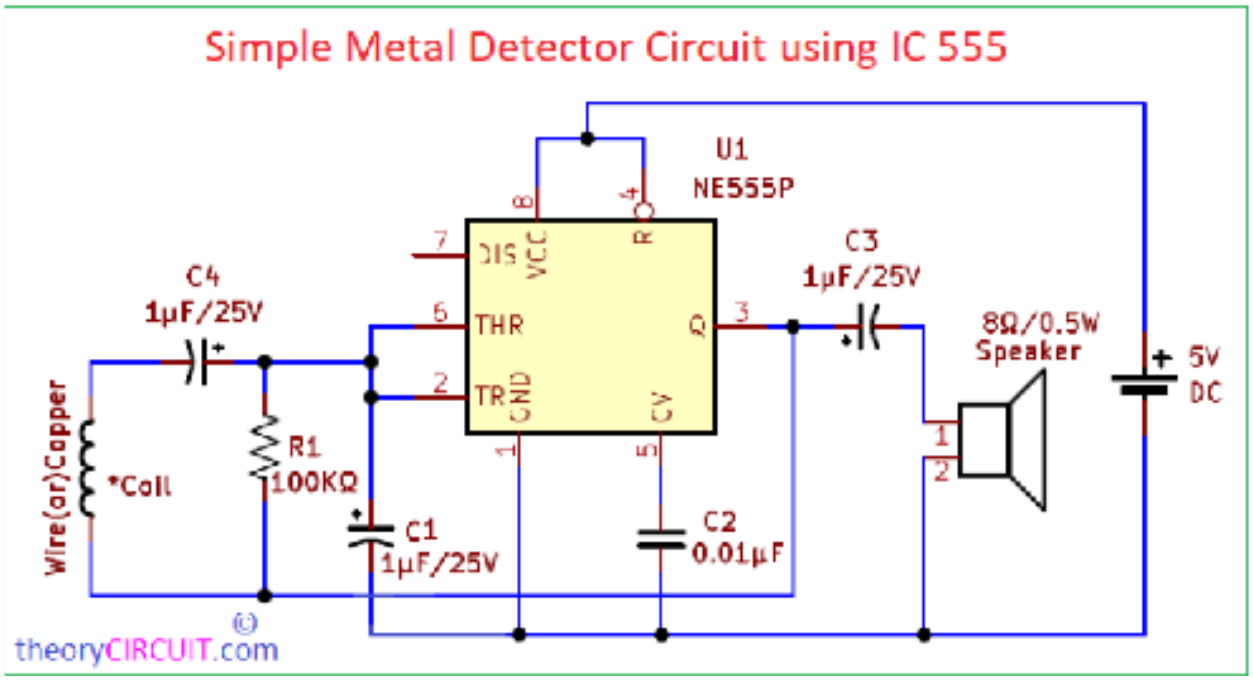

Figure 2: The metal detection circuit

The circuit functions by sending a signal of a certain frequency through the coil and the speaker. When the coil is held near metal, the inductance of the coil changes, changing the frequency of the signal, causing the tone the speaker emits to shift. After breadboarding this circuit and ensuring it works, we decided to make some changes to comply with the metal detection guidelines. We swapped out the AC buzzer for a header that connected to the STM so it can read the frequency of the signal directly. The STM was then connected to a DC buzzer and would power it if the frequency of the signal was above a certain threshold frequency. This would keep the buzzer from emitting sound unless metal was detected. To further improve this circuit in the future we could use the STM to generate a signal instead of the 555 timer to cut down on the amount of components on the circuit board. The implementation of the metal detection circuit in the PCB schematic is shown in Figure 3.

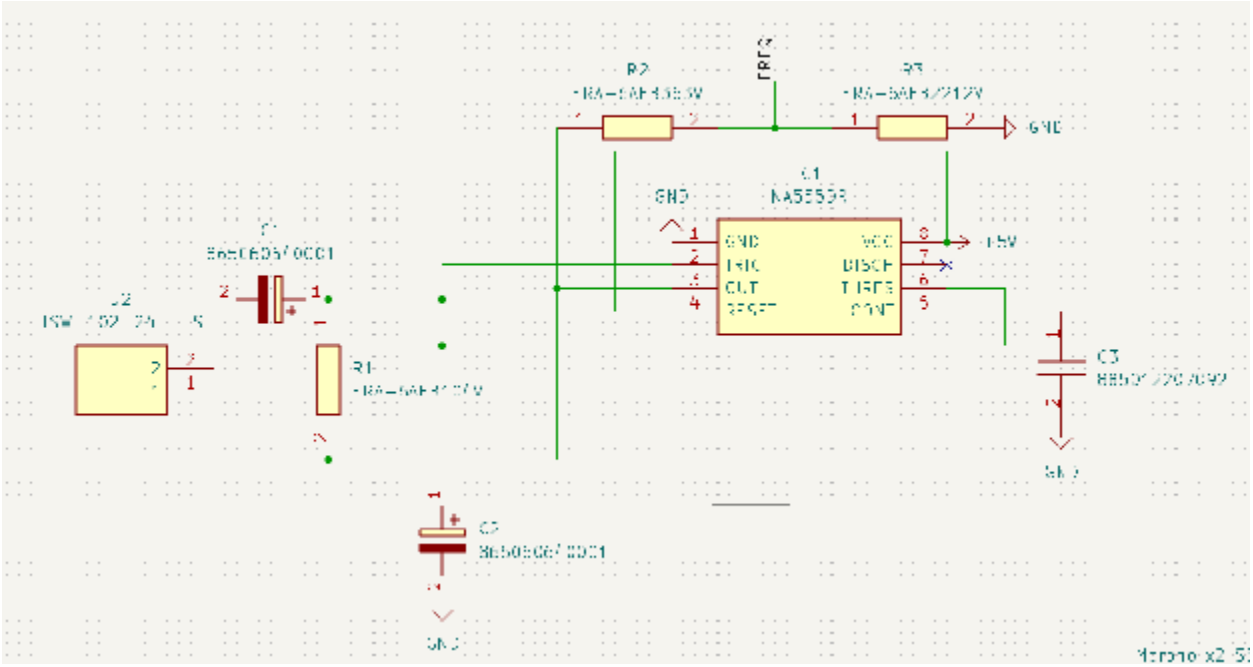

Figure 3: The PCB schematic for the metal detection circuit

After breadboarding and testing this new circuit worked, we made a PCB for this circuit that would also hold the STM that we could mount to the car. The final hurdle that needed to be cleared was the detection coil. Over three iterations in total, it was discovered that the denser the windings of the coil, the greater the change in frequency and therefore the greater margin for error we had when setting up the threshold frequency. This was especially important because the baseline frequency was prone to drifting up and down during testing. We also found that we would get better results the closer in diameter the coil was to the copper plates.

The distance sensing component of the project is a simple connection between the STM32 MCU and the ultrasonic sensor module, using GPIO pins to trigger and read signals from the sensor.

Originally, for the motor driver circuit we used the L298N which was given to us in ENGG 200 as a place holder for the PCB board. We decided to place this circuit onto a PCB board to fulfill the PCB requirements for the project. For the motor driver circuit, we decided to use the circuit shown in the technical workshop as we had already built this one for the workshop. A design iteration that we did explore for this PCB board was soldering the esp32 module right onto the board, but this limited our access to the esp32 with a cable. We switched this to an esp32 devboard which slotted in with pin headers and allowed us to switch the esp32 if needed. Also, this PCB board was switched from a 2 layer to a 4-layer board to allow for better power dissipation and more room for copper pours. To validate this PCB board, we first checked to see if there were any shorts on the board, then tested the outputs on the buck convertor, then tested our motors to see if the circuit worked. Once soldered correctly, the circuit board worked on the first iteration and performed as expected in simulations. The temperatures of the motor driver's ICs were also monitored to ensure there was proper heat dissipation.

3.3 Power Management

On the vehicle, there are three voltage levels which are 12V, 5V, and 3.3V. We decided to use 12V because this would allow us to use higher voltage motors, which would ensure we have enough power to make the steep climb. The 5V power rail was used to supply power to the voltage regulator on the esp32 board and power the stm32 microcontroller board. The 3V3 power rail was used to power the esp32 microcontroller and provide a high logic level to the motor driver ICs.

The battery was a 12V 2800mAh rechargeable battery which supplied more than enough power and current for our vehicle. With the total capacity of the battery being 2800mAh and estimating an average current draw of 500mA, this would put our runtime at 5.6 hours.

The maximum power draw for each motor is 13.54W according to the datasheet. The motor driver ICs chosen can provide over 40W of power which means we are supplying enough power to the motors. At 12V, the motors would have a stall current of around 1.1A and our battery is able to supply this amount of current while our motor drivers can handle up to 3.5A.

For circuit board protection, there is a fuse to limit current and reverse polarity protection. For the fuse, it is a fuse blade so if the fuse breaks, we can switch it out if needed with ease and it is set at 5A which is the maximum current our battery can provide. The reverse polarity protection can prevent the board from damage if the battery is plugged in the incorrect way.

3.4 Printed Circuit Board Design

The functions integrated on the first printed circuit board design include a buck converter to turn the 12V from the battery to 5V, integrated circuit protection, and a motor driver circuit. For this printed circuit board, there are 5 2-port screw terminals to allow for high current flow and a secure connection. Two of these ports are used to supply the motors with current, one of the ports is used to plug in the battery, one of the ports is used to plug in our led headlights, and the last port is used to supply a 5V supply for our stm32 board. Another connection interface used for the PCB board was 2 20-pin headers to connect our esp32 board to the PCB board.

For thermal design considerations, I decided to use a 4-layer board to allow for a power plate that contained copper pours for 12V power, 5V power, and 3.3V power. Two of the plates were used for signal tracing, and the final plate was used as a ground copper pour. I also decided to use copper pours by the fuse into the circuit protection as this would be where most of the current is flowing into the circuit board. To allow for power dissipation, I used 0.8mm vias and placed extra vias on the larger pads and the ground pad of the motor driver ICs. For layout decisions, I decided to place the battery protection pieces at the bottom of the board where the battery is connected. I decided to place the motor driver's ICs on the left and right side of the board and the buck converter near the middle of the board. This allowed me to keep the high heat pieces separated from each other and allow for space for each circuit. I also used thick 2mm traces for high current routes and copper pours for each of the power voltage connections.

The PCB schematic and layout for the board handling the ESP32 and movement are shown in Figure 16 and Figure 17, as well as a 3D rendering in Figure 18.

The PCB designed for the STM32 MCU and the metal detection and distance sensing components is much simpler, as there are no high power components. A number of header pins are used to route the STM32 pins to their respective components, while the main metal detection circuit is

contained on the board. The PCB schematic and layout for the board handling the STM32 and these components are shown in Figure 19 and Figure 20, as well as a 3D rendering in Figure 21.

3.5 Remote Controller Design

The remote controller was designed using a software approach through a web server rather than a physical hardware design as we wanted to reduce the cost factor of a physical controller. This not only provided us with full customizability of the controller during testing to see what design felt comfortable but also allowed us to focus more on other hardware designs for the car as the user interface was somewhat predetermined with the use of a phone. Furthermore, the user layout was created with the layout of a simple RC controller that utilizes one joystick for the throttle and another for the steering. On our web server, we include one vertical slider om the left-hand side that is purely for forward and reverse throttle, and a horizontal slider on the right-hand side for steering of the car. These sliders both had an included dead space around the center to allow for easy stoppage or straight driving of the vehicle. Additional buttons were added to the controller, specifically to toggle the LED's and reset the sliders to their base position at zero.

Our controller utilizes the ESP32's built in WIFI functionality as a form of communication and accesses an HTTP web server over this network. Essentially, the user will connect to the WIFI network with their phone and enter the specific ESP32 IP address into their web browser (Safari works best). Once the user provides an input, the server sends an HTTP GET request to the ESP32 which is handled in the uploaded code on the microcontroller, which responds with the action tied to that command. The data is encoded with HTTP URL query strings where certain parameters, like throttle and steering, are paired. Other interactions like the use of the LED or stop buttons immediately access the endpoint commands. When the ESP32 receives this data, the handler functions defined in the code parse the data to extract its basic parameters and convert them to the values needed for the PWM speed and direction of the motors for example. This strategy allows our controller to provide immediate feedback to the motors and LED.

The remote control is tied to the motor PCB board and interacts with each of the motors through the motor drivers on the board. When the user's input is processed, the ESP will send a voltage to the motor drivers on the board, or to the LEDs mounted on the front of the car with a simple parallel LED circuit.

3.6 Software Architecture

The software block diagram for the movement systems of the design is shown in Figure 4.

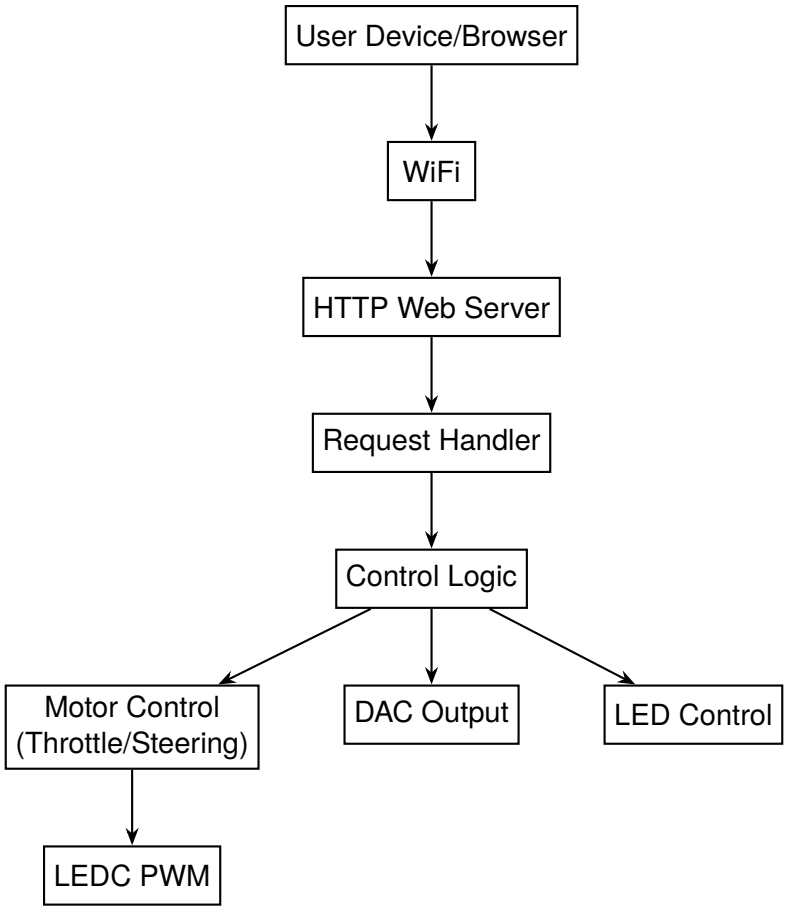

Figure 4: Software block diagram for the movement of the design

The software block diagram for the STM32 MCU is shown in Figure 5 and Figure 6.

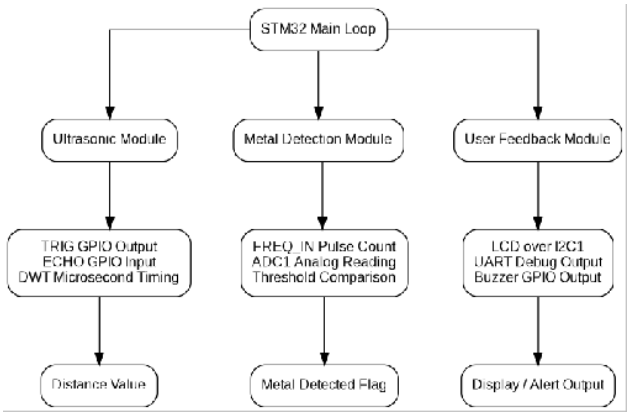

Figure 5: Software block diagram for the STM32 (1)

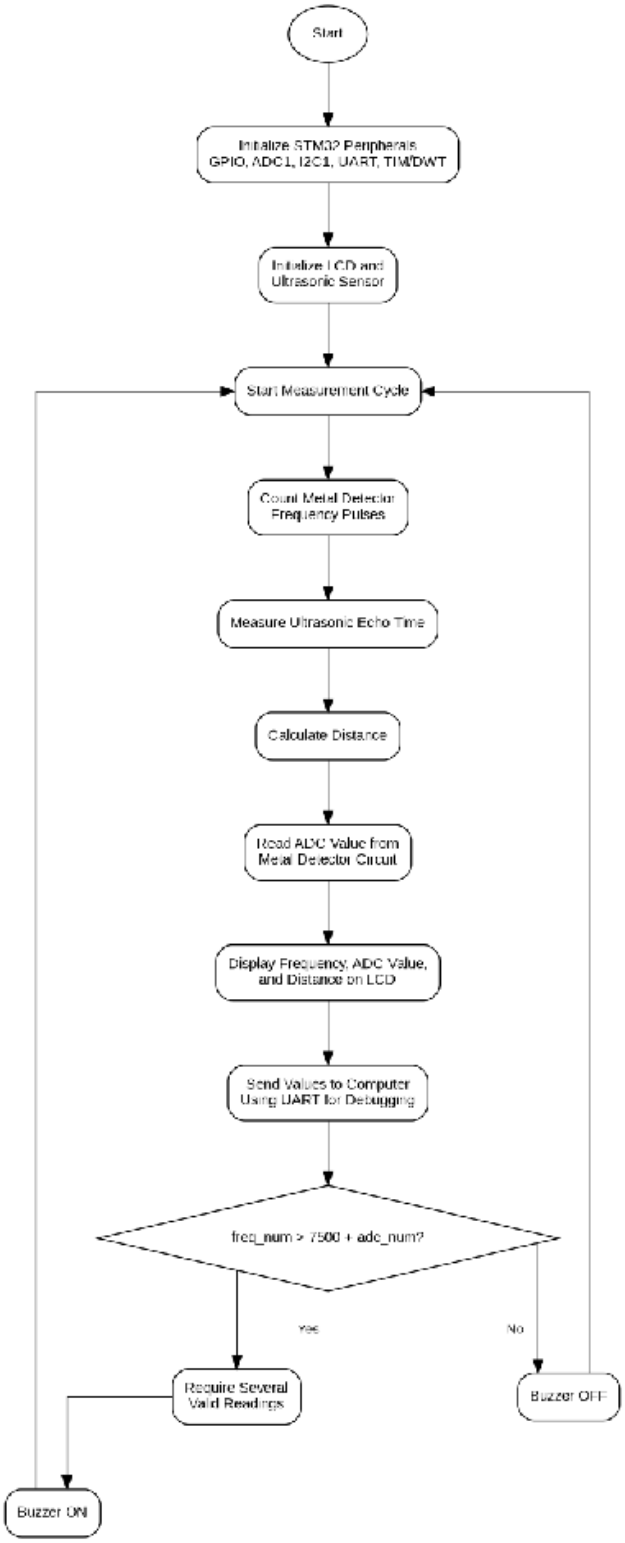

Figure 6: Software block diagram for the STM32 (2)

3.7 Software Design

Our vehicles design includes both the STM32 and the ESP32-Wroom-32E microcontrollers. Our deviation from the STM32 was made mainly for our controller design as we wanted it to be virtual and through WIFI. Additionally, the ESP32 allowed for more space on our car, and we overall felt more comfortable working with the ESP32 and its code environment. The software used with the ESP32 was through the ESP-IDF extension on Visual Studio Code (VS Code) and the main peripherals that were configured were GPIO, LEDC, WIFI, and DAC. GPIO pins were assigned as output to control the motor directions and to toggle the LED output. Input GPIO pins were defined for current sensing applications as well, although we did not use them in our final iteration. Moreover, the LEDC peripheral was used to control the speed of the motors through PWM. Our motors were configured at a frequency of 5kHz and an 8-bit resolution. Additionally, a DAC channel was used to allow for easy testing of our metal detection circuit as it could adjust the baseline frequency used to read the copper plate. This was a necessary step for testing our metal detection circuit as our original circuit design was inaccurate and allowed us to more easily determine the issues with our circuit like cold solder joints and too few copper coil turns. Lastly, WIFI was configured to provide a wireless network signal for the user's device to access the web server and control the car. Communication between the remote control and the ESP32 was defined using WIFI and HTTP requests to the microcontroller. Through the wireless network that the ESP32 hosts itself, a web server can be accessed that receives HTTP requests from the user. These requests are handled and processed to output the desired action.

Four modular components were defined for this project: these include the main module designed specifically for initialization of the system, a motor module in charge of controlling the actions of the motors like speed through PWM and direction, a web server module responsible for the user interface of the controller as well as processing the HTTP data from the user's inputs, and lastly a WIFI module that creates the wireless network for the user to connect to. Each of these modules had specific print functions defined for debugging and could be seen in the terminal when the ESP32 was connected to the computer. These were mainly used to check data transmission for our sliders to see the HTTP requests in real time and to access and update our controller interface. Additional debugging was done by testing each component separately, which was mainly done for the motors to adjust for speed inaccuracies caused by slight differences in the motors, gearboxes, and friction.

Our system requires the use of immediate responses when it comes to driving the car as quick actions are needed to adjust the car quickly in tiny areas to avoid crashes and properly read for distance or metal. We achieved this using our web server and HTTP GET requests through our sliders that update the outputs to the motors. For example, any time you slide the throttle slider up past the center dead zone to make the car move forward, an HTTP GET request is sent to ESP32 for every change in the sliders position. Because it is an 8-bit resolution for the PWM speed, the ESP32 can receive up to 255 requests with this one action and translate each of those requests to motor actions almost immediately. Similarly, this same process is used for the LED and stop buttons. This was important for us to use as we wanted there to be little latency between the controller and driving control and make it relatively straightforward to operate with the layout of our controller.

The key role of the STM32 was distance sensing and operating the metal detection circuit. It was used as the dedicated sensing and data-processing controller for the RC vehicle. While the micro-controller has many different pins and possible configurations, the main peripherals configured for

this project were GPIO, ADC1, I2C1, USART2, TIM2, and DAC resources using STM32CubeIDE. The GPIO pins were configured as input/output pins for the ultrasonic trigger and echo signals, which are both digital sensor interfaces. In addition, GPIO pins were also used for the buzzer output and the frequency input from the metal detection circuit. Furthermore, ADC1 was configured to sample the output of the metal detection circuit so that the microcontroller could monitor signal amplitude and use this information in the detection logic. Since an LCD screen was used to display the measured distance and metal detection frequency, I2C1 was configured to allow communication between the microcontroller and the display. Lastly, USART2 was used for serial debugging, where live sensor values were frequently transmitted back to the computer during testing and development.

The STM32 was programmed to generate a 10 µs trigger pulse to control the ultrasonic sensor. After receiving this pulse, the sensor emitted an ultrasonic wave and waited for the reflection from a nearby object. The program measured how long the echo signal remained high, and from that value, it calculated the distance to the object. This method worked well for the vehicle because it provided accurate measurements while the car was in motion. At the same time, the STM32 was also programmed to measure the signal frequency and ADC value from the metal detection circuit. The frequency was obtained by counting pulses over a period of time, while the ADC measured changes in signal level. Both values were used to create a threshold for detecting the presence of metal beneath the vehicle. Since the buzzer could be triggered too easily by noise or false readings, the STM32 was programmed to require several similar successful readings before turning the buzzer on. This improved reliability and reduced random false alarms. For user feedback, the LCD screen displayed the ADC value, measured frequency, and distance to nearby objects. This allowed the team to verify that the system was operating correctly. In conclusion, the STM32 was responsible for sensing and detection tasks, while the ESP32 was used for motor control and communication between the phone and the vehicle.

3.8 Mechanical Architecture

Two distinct mechanical designs will be discussed. The first design (see Figure 7 and Figure 8) was the initial concept implemented, later redesigned to the second design shown (see Figure 9 and Figure 10) in response to certain challenges related to the mechanical system observed throughout the development of the project. For the following diagrams, the housing for the gearbox on one side is omitted to show the gear reduction used in each design.

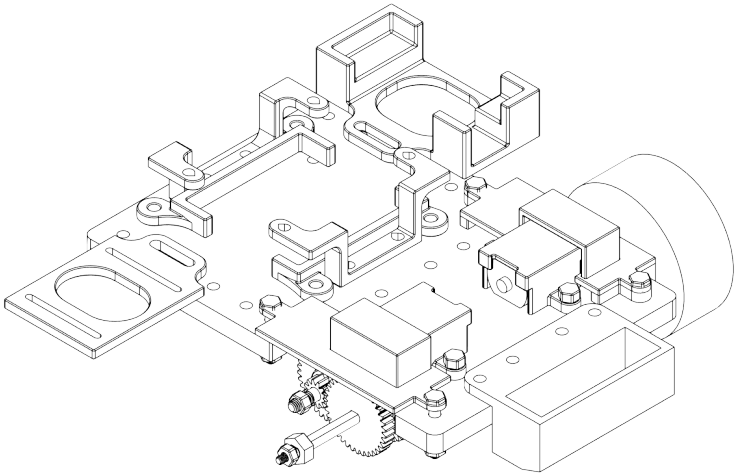

Figure 7: Drawing of the initial design (1)

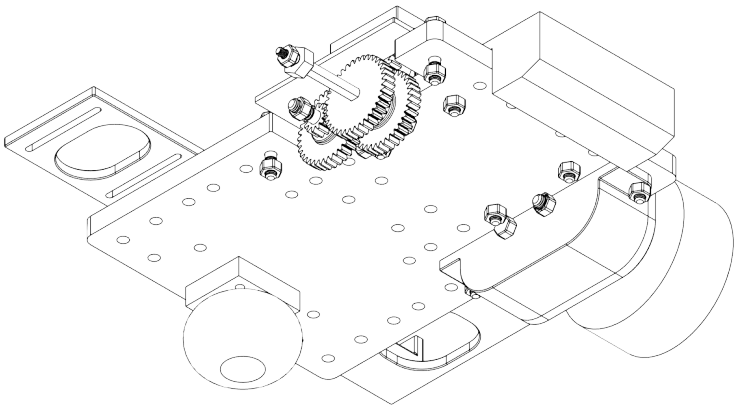

Figure 8: Drawing of the initial design (2)

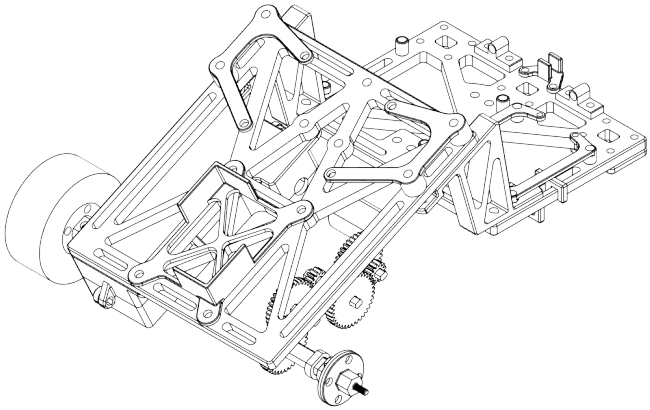

Figure 9: Drawing of the final design (1)

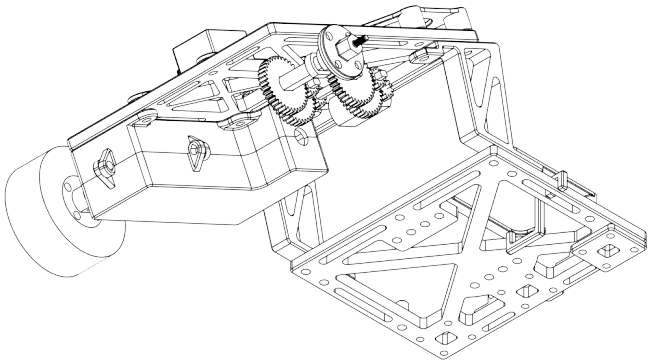

Figure 10: Drawing of the final design (2)

The mechanical block diagram for the initial design is shown in Figure 11.

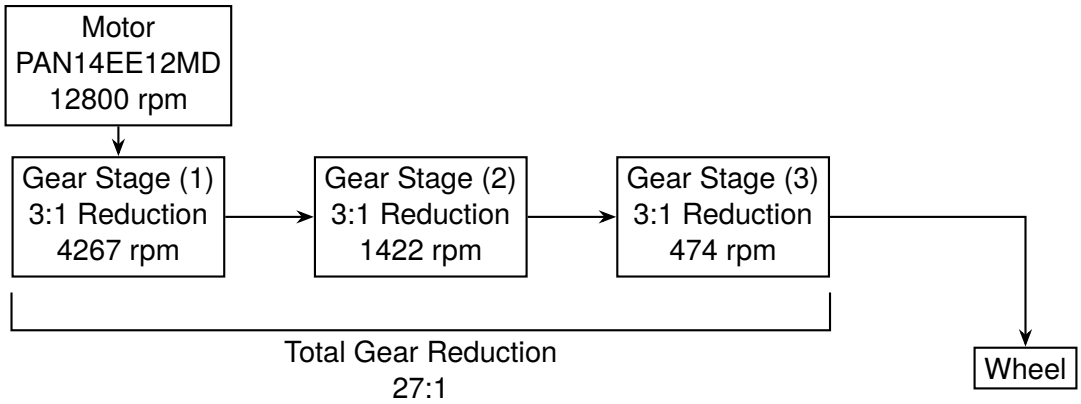

Figure 11: Mechanical block diagram for the initial design

The mechanical block diagram for the final design is shown in Figure 12.

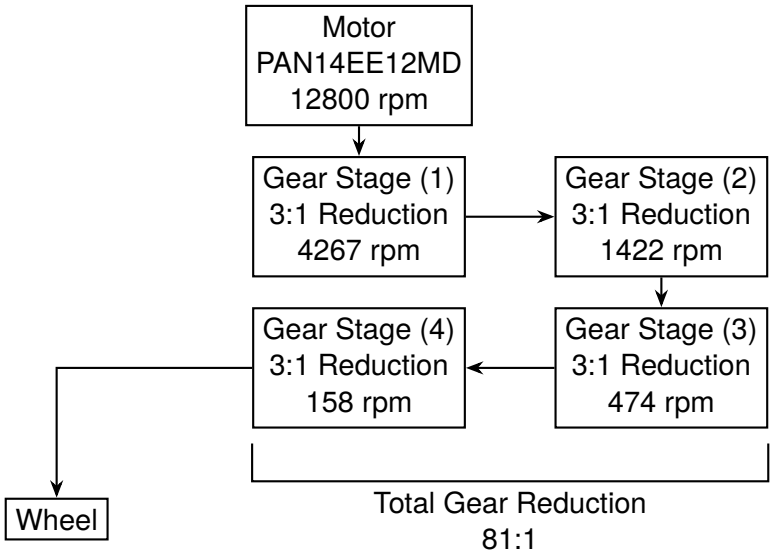

Figure 12: Mechanical block diagram for the final design

3.9 Mechanical Design

The drivetrain design involves a tank steering system, where two independent motors are controlled to be able to turn and steer the vehicle by varying the speed of the wheels, that is, speeding up one wheel and slowing down the other. This design was chosen due to certain advantages compared to other options, such as using one motor to drive a differential and thus both wheels simultaneously. With tank steering and the usage of two separate motors, the mechanical complexity is lower, thus reducing the chance of failure within the mechanical system.

An 81:1 total gear reduction is implemented through 4 stages of 3:1 gear reduction to reduce the nominal rpm of the motors to 158 rpm, enabling finer control of the vehicle. Furthermore, this creates the necessary torque to climb the ramp through mechanical advantage.

The chassis design involves two separate frames, divided to be able to 3D print the chassis within

the size of the print bed. These pieces are combined through the use of nuts and bolts. The bracket between the two frames creates the necessary clearance to clear the top lip of the ramp. The beams are intended to provide structural strength and integrity. The use of these beams compared to a solid block, as seen in the initial design (Figure 7), allows less material usage, faster print times, and reduced weight in the final vehicle. A number of mounting holes are placed throughout the design of the chassis to facilitate the attachment of the necessary components, such as the gearboxes, PCBs, ultrasonic sensor, and LCD display screen.

The primary material used in the implementation of the vehicle is PLA through 3D printing, including all structural components such as the chassis and gearbox housings, the mounting brackets for electric components, and the gears for the drivetrain. PLA was chosen for its ease of access, printing, and availability, as well as its generally favorable mechanical properties (rigidity).

Components are mounted to the chassis primarily using nuts and bolts, with custom mounting brackets for each component being designed and printed in PLA. An exception to this the attachment for the ultrasonic sensor, mounted by using a clip-style fit that is then bolted to the chassis, for simplicity and avoiding designing around the much smaller mounting holes of the sensor. The motors for both designs are held within an enclosure, constraining their movement. These mounting strategies were chosen for availability, ease of design, and more secure mountings.

A physical challenge encountered during the design of the vehicle involves the limitation of the bed size of a 3D printer, which limits the maximum size of a printed component in one piece. Thus, as seen in the final vehicle, it was necessary to split the frame into multiple pieces to be assembled later. Another constraint was the need for an ample amount of space for mounting components, exacerbated by the large gearboxes relative to the other components. As well, the final size of the vehicle was considered throughout the design process to stay on the ramp and avoid falling off, as well as having the ability to navigate the obstacle course effectively, as a larger vehicle would be harder to control. This is alleviated by using a large gear reduction, allowing a range of rpms lower than normal, allowing finer control.

The iterations through designs are most apparent when comparing the initial and final designs. A key revision is the addition of a 3:1 gear reduction stage to the gearbox, reducing the rpm of the wheels from 474 rpm to 158 rpm. This enables finer control and movement of the vehicle, albeit slower though this is largely a desirable result. Furthermore, this reduction in speed and increase in torque ensures the ability to clear the ramp and reduce the amount the motors stall. The gearboxes were also redesigned to accommodate the change in drivetrain and addition of another gear stage, and the motor was moved to be within the gearbox as well. Another key change from the initial to the final design is the redesign of the chassis, where it was split into two pieces, larger in total size than the original chassis. This created more space for the mounting of components and the ability to attach the larger gearbox. As well, the final chassis is changed from a solid block with mounting holes to a frame-like structure, keeping approximately the same mass and material usage while creating a larger total area. A notable change is the relocation of the ultrasonic sensor from the side of the vehicle to the center axis, as in testing it was notably difficult to aim the sensor at a target when it is mounted to the side. The center of mass is also moved towards the center of the vehicle, as in the initial design the battery was mounted at the very end of the vehicle behind the wheels, while the final design has the battery above and slightly forward from the wheels. This creates a better weight distribution that still keeps a majority of the weight on the wheels, enabling better traction and control. A more minor revision is the addition of a second caster wheel, reducing the amount of roll the vehicle experiences, albeit this was unlikely to drastically affect the performance of the vehicle. The implementation of the final design in shown in Figure 13, Figure 14, and Figure 15.

The primary tests used to validate mechanical performance were trials testing the ability of the vehicle to clear the ramp, where it was able to perform effectively in both designs. However, in the initial development and testing of the final design, which involved a flat chassis without the bracket, the car would clip the top lip of the ramp, as previously discussed, and thus was unable to complete the ramp. This was solved by the brackets which increased the clearance under the car and allowed the car to complete the ramp. As well, the maneuverability of the vehicle was tested by using the ultrasonic sensor and metal detection circuit to detect their respective targets, as would be done in the final obstacle course. In the initial design, the vehicle was found to be difficult to control and effectively detect the targets. The higher speed added difficulty to steering and overall maneuverability. This was remedied in the final design with the addition of a gear stage, reducing the speed once more.

Something that could be improved with the vehicle is the overall size, as compared to other designs it is observed to be somewhat larger. This would better the maneuverability of the vehicle and reduce the total weight of the vehicle further. A possible solution is the usage of motors with an internal gearbox, as metal gears provide the necessary strength and precision at that scale when compared to 3D printed gears. This would reduce the size of the gearboxes and most likely the amount of gear stages, which occupy a large amount of the vehicle volume.

4 References

[1] Espressif Systems, ”ESP32-WROOM-32E & ESP32-WROOM-32UE Datasheet”, Version 2.0, Datasheet, 2021. [Online]. Available:
https://espressif.com/documentation/esp32-wroom-32e_esp32-wroom-32ue_datasheet_en.pdf

[2] Team Analog, “Simple metal detector circuit using IC 555,” TheoryCircuit, Feb. 28, 2024. [Online]. Available:
https://theorycircuit.com/ic-555-ic-741/simple-metal-detector-circuit-using-ic-555/

[3] STMicroelectronics, “STM32F446xC/E datasheet,” STMicroelectronics, 2016. [Online]. Available:
https://www.st.com/resource/en/datasheet/stm32f446mc.pdf

[4] Sysprogs, “STM32 HAL libraries and samples,” SourceVu, [Online]. Available:
https://sourcevu.sysprogs.com/stm32/HAL/

5 Appendices

Table 1: Bill of materials

Item	Item Description	Source	Unit Price	Quantity	Cost
1	Wheels	Amazon	$5.00	2	$10.00
2	Motors	DigiKey	$9.16	2	$18.32
3	ESP32	Amazon	$13.00	1	$13.00
4	Battery	Amazon	$25.99	1	$25.99
5	PCB parts	DigiKey	$31.80	1	$31.80
6	Caster wheels	Home Depot	$4.00	2	$8.00
7	Bolts and nuts	Home Depot	—	—	—
				Total	$107.11

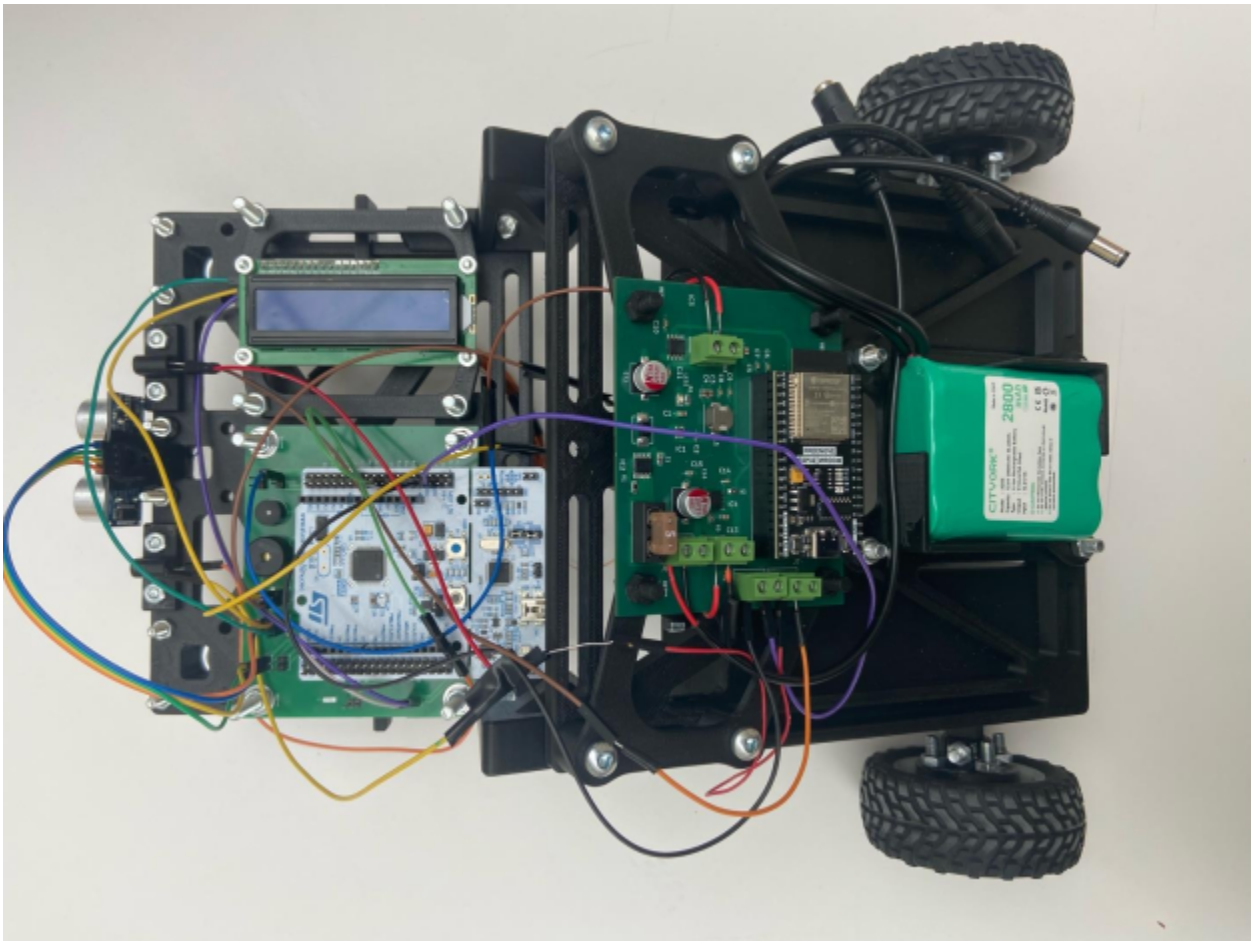

Figure 13: Implementation of the final design (1)

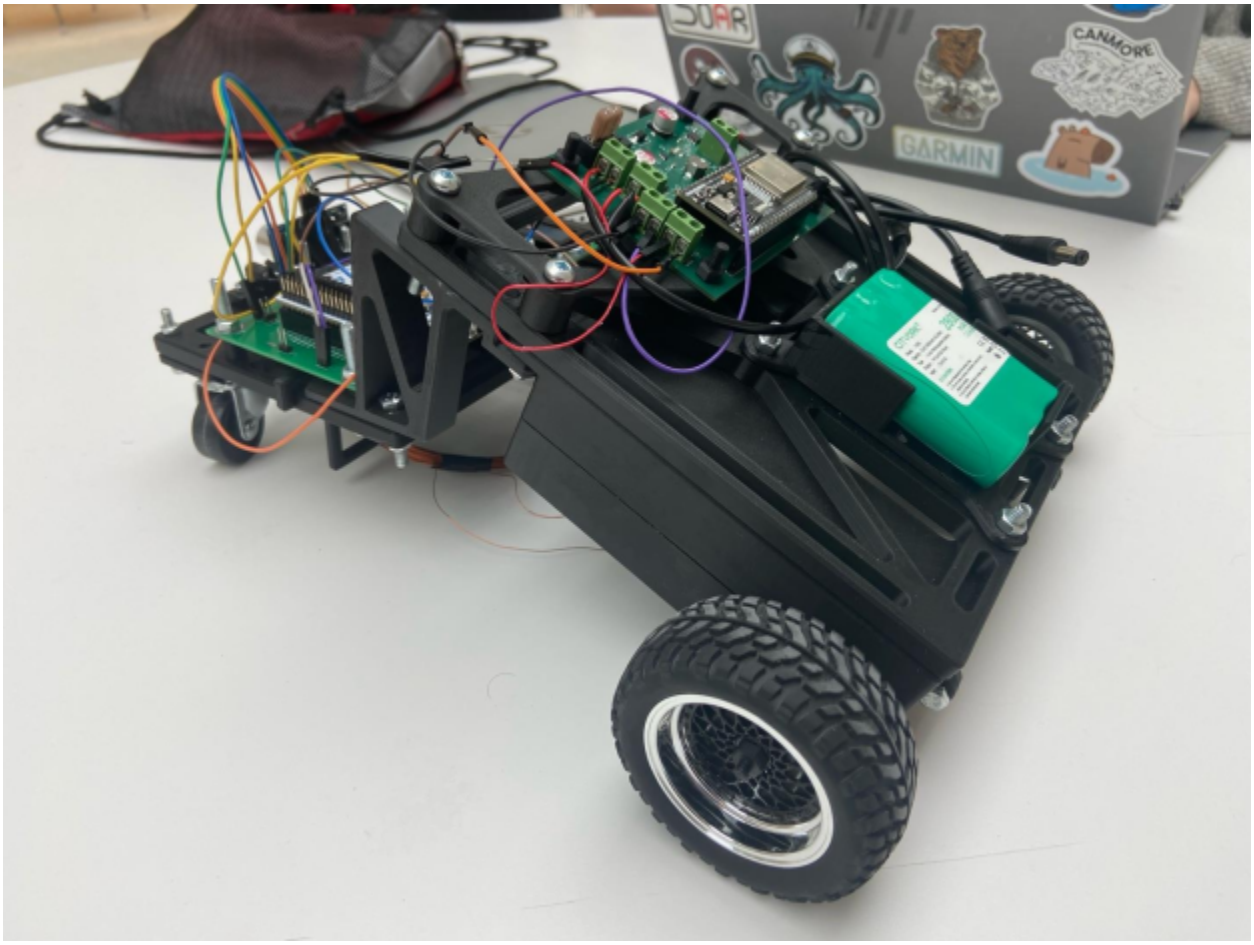

Figure 14: Implementation of the final design (2)

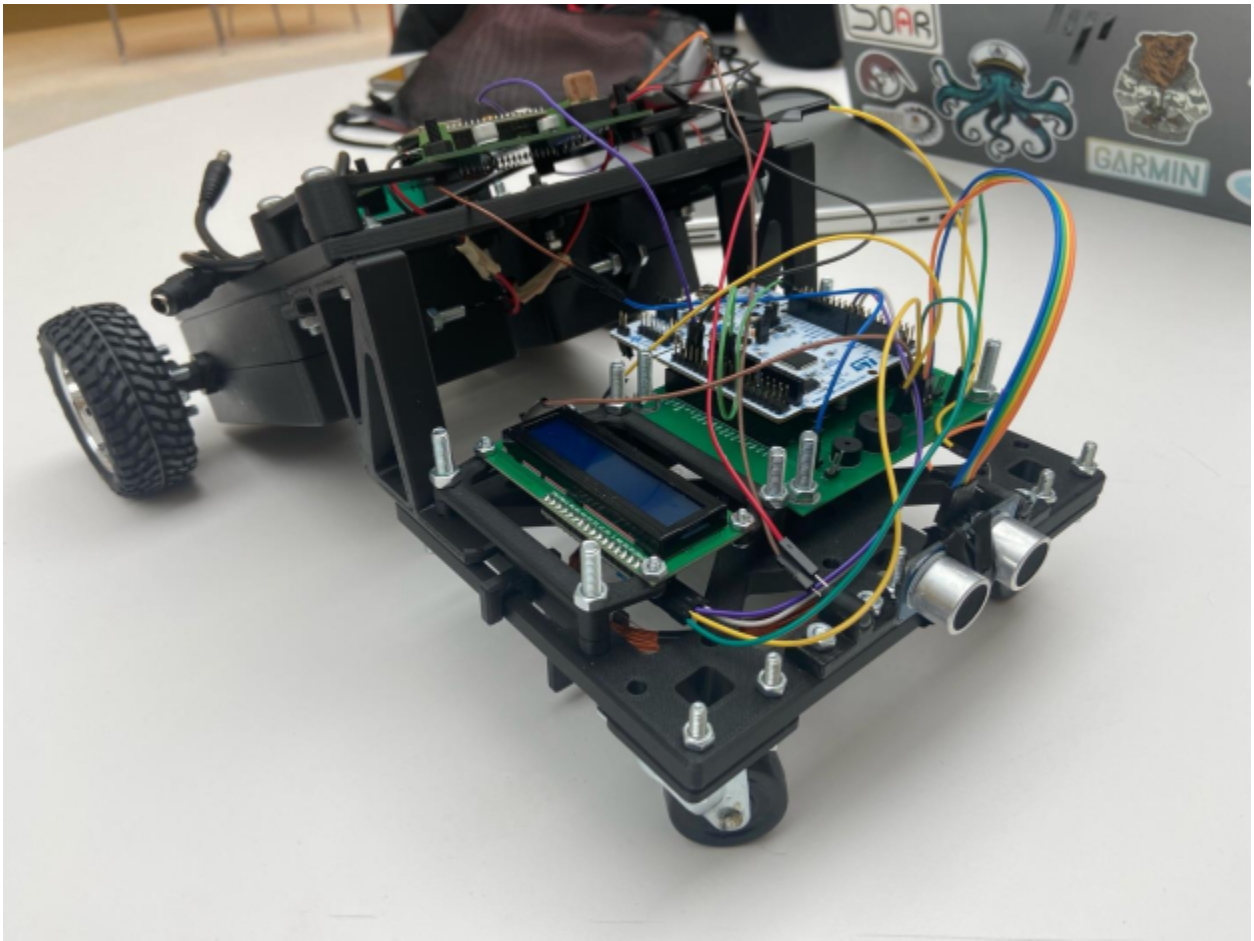

Figure 15: Implementation of the final design (3)

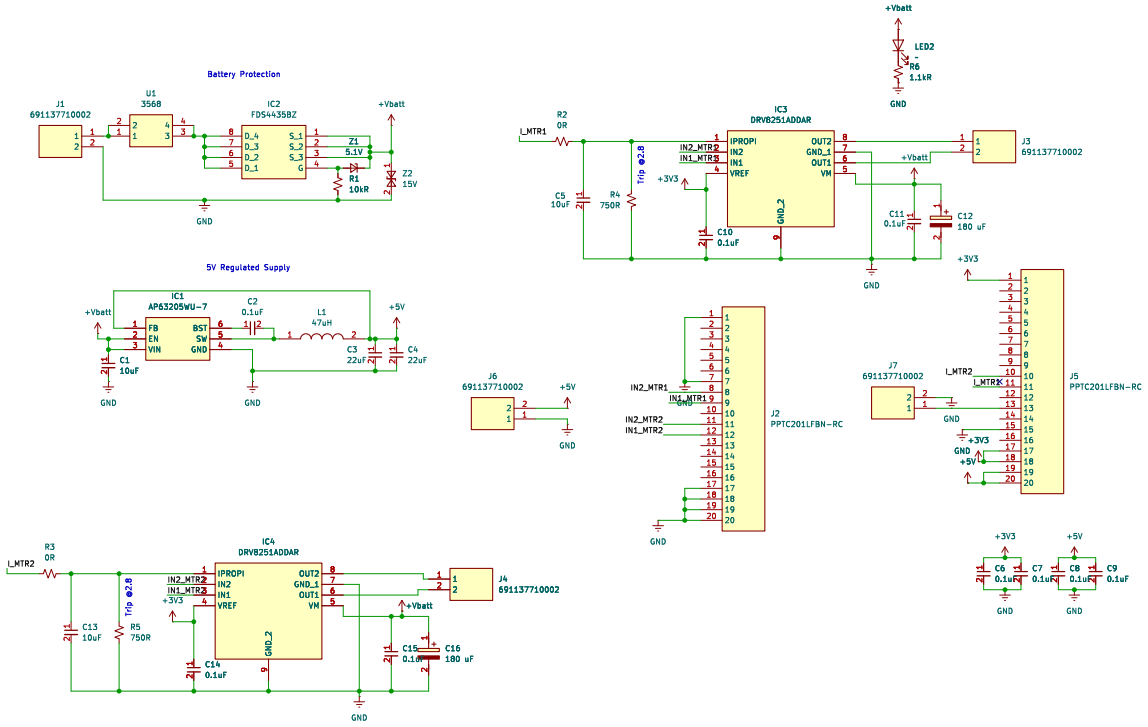

Figure 16: Schematic for the ESP PCB

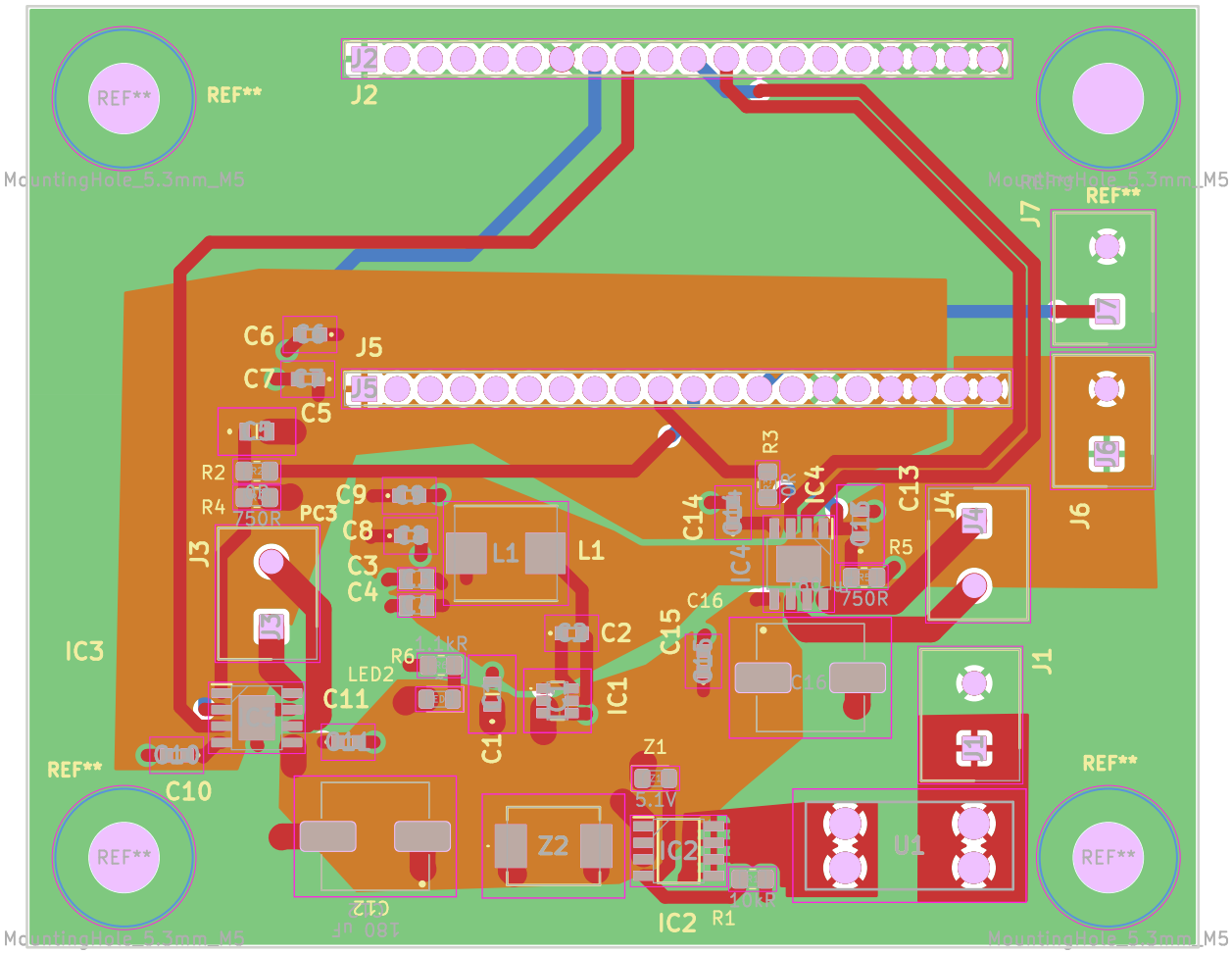

Figure 17: Layout for the ESP PCB

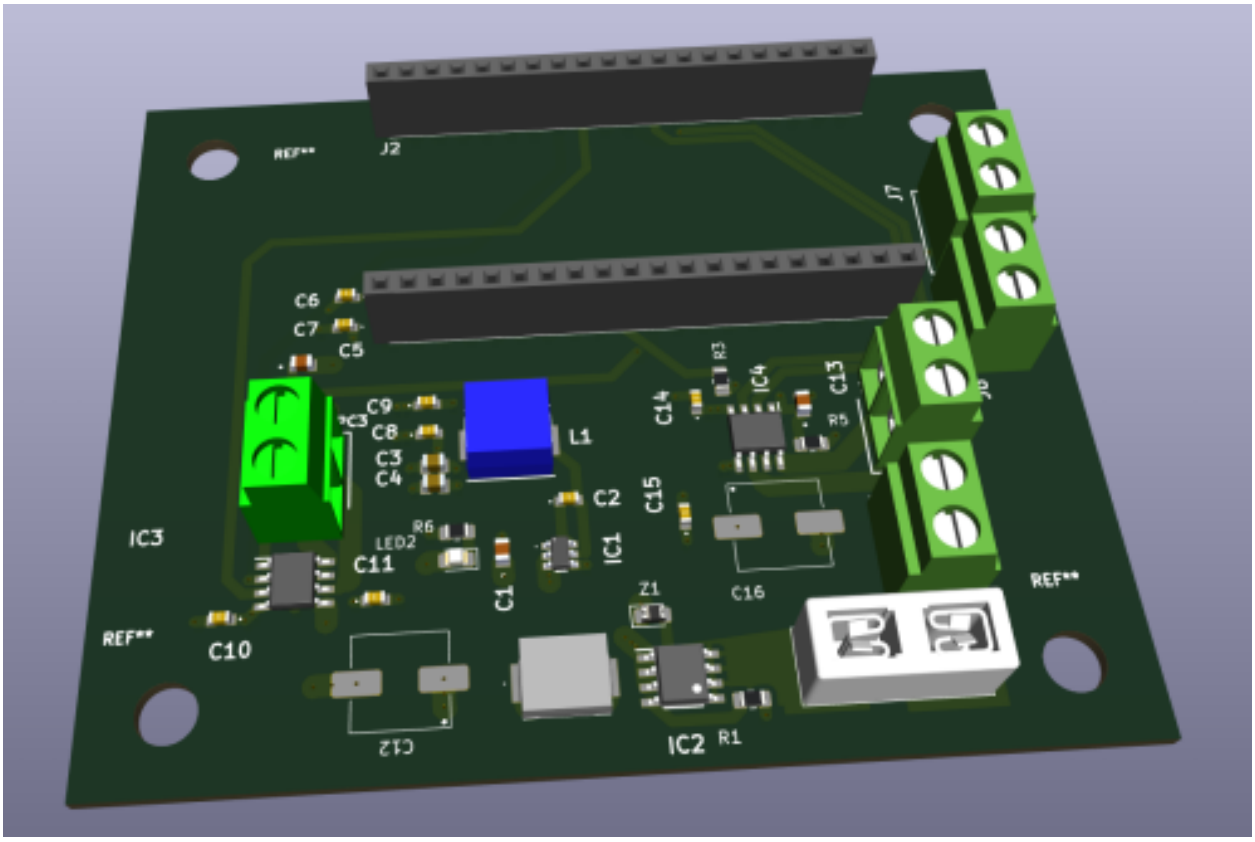

Figure 18: 3D rendering of the ESP PCB

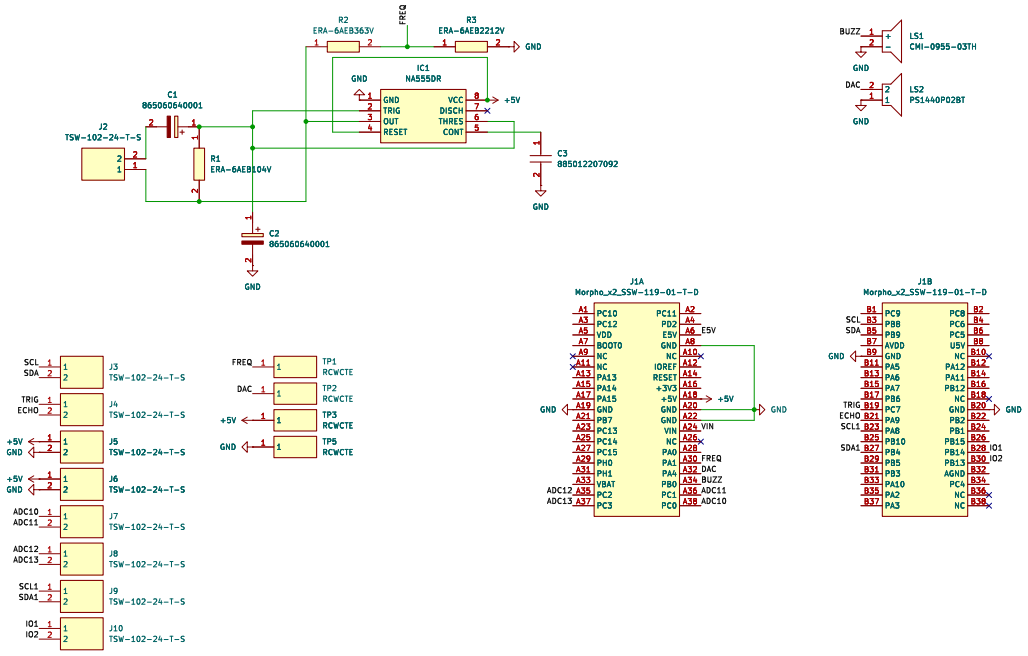

Figure 19: Schematic for the STM32 PCB

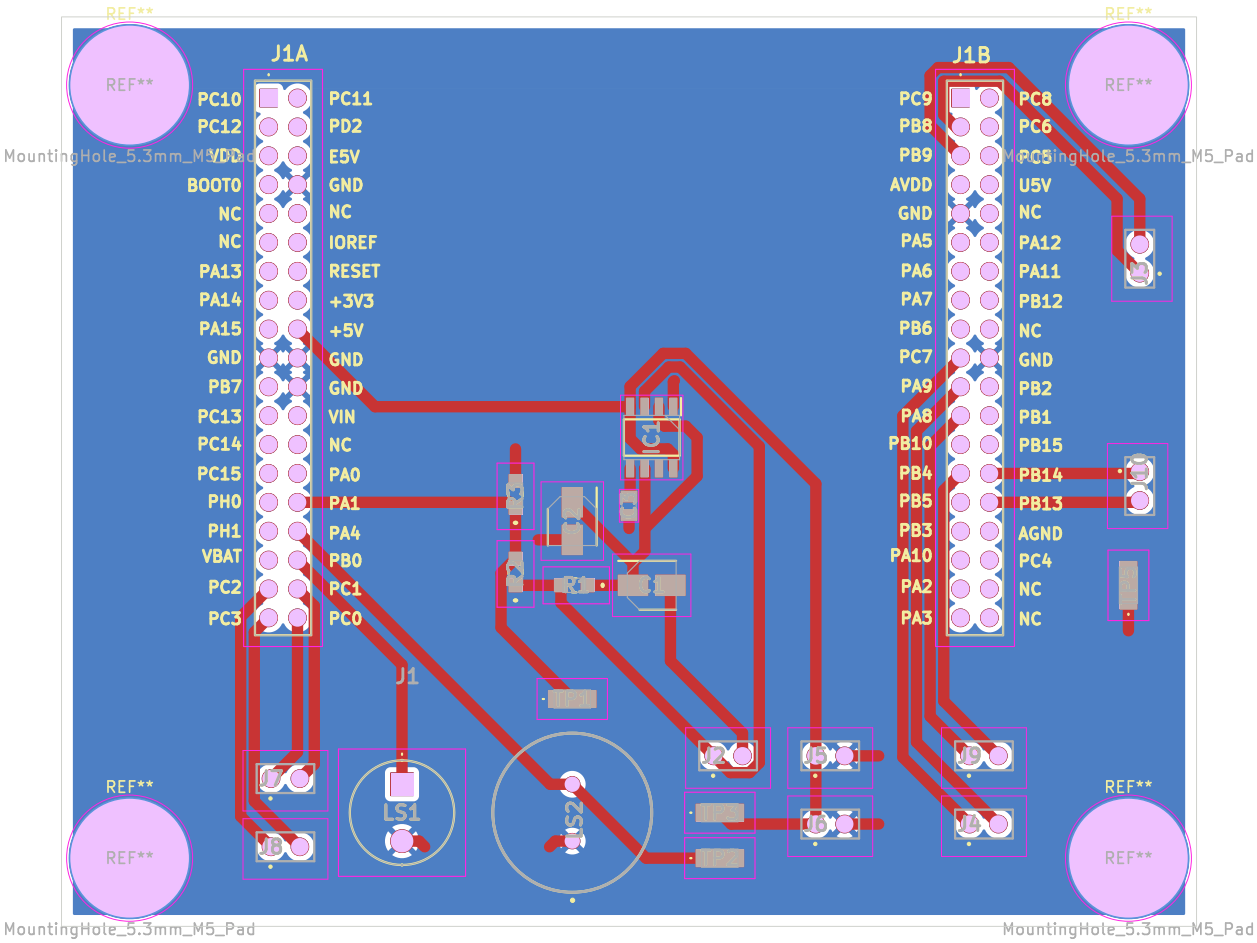

Figure 20: Layout for the STM32 PCB

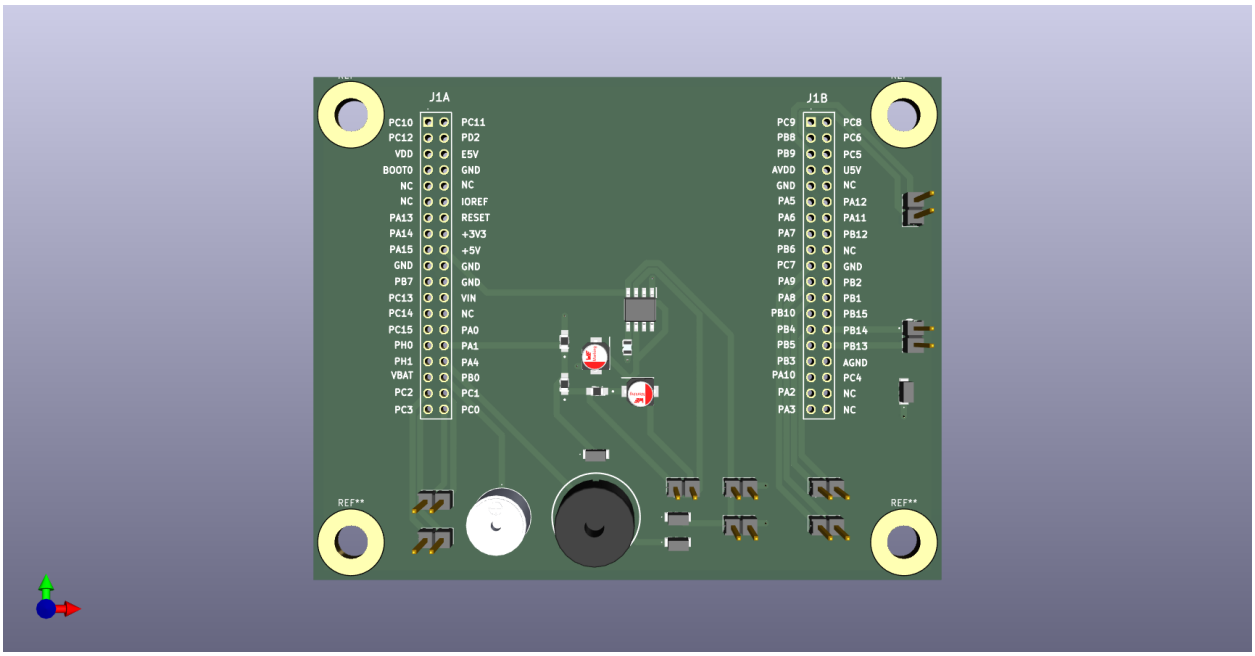

Figure 21: 3D rendering of the STM32 PCB